💻 SQL Querying – Functions, Joins, and Optimization Techniques

8/31/2026

1. Basic Querying & Table Structure 🍕

Selecting Columns

- Use SELECT to retrieve specific columns from a table.
- Example: Selecting pizza_type_id, pizza_size, and price from pizzas table.

```
SELECT pizza_type_id, pizza_size, price FROM pizzas;
```

Filtering Data with COUNT

- COUNT(*) counts total rows.
- Use WHERE clause to filter rows before counting.
- Example: Count total pizzas vs. count pizzas in 'Classic' category.

```
SELECT COUNT(*) AS total_rows FROM pizza_type;

SELECT COUNT(*) AS total_rows FROM pizza_type WHERE category = 'Classic';
```

Checking Table Structure

- Use DESC TABLE or DESCRIBE TABLE to view columns, data types, default values, and nullability.
- Example: Inspecting orders table structure.

```
DESC TABLE orders;
```

---

2. Data Type Conversions & Formatting 🔄

- Two methods for type casting in Snowflake:
  - CAST(column AS type)
  - Shorthand: column::type
- Example: Convert order_id to string (VARCHAR), price to numeric (NUMBER).

```
SELECT CAST(order_id AS VARCHAR) AS order_id_string FROM orders;

SELECT price, CAST(price AS NUMBER) AS price_dollars FROM pizzas;
```

3. String Functions ✍️

Function	Description	Example Usage
INITCAP()	Capitalizes the first letter of each word	INITCAP(pizza_type_id)
CONCAT()	Concatenates multiple strings	CONCAT(name, ' - ', category)

Examples:

```
SELECT INITCAP(pizza_type_id) AS capitalized_pizza_id FROM pizza_type;

SELECT CONCAT(name, ' - ', category) AS name_and_category FROM pizza_type;
```

4. Date & Time Functions 🗓️ ⏰

Function	Purpose	Example Usage
CURRENT_DATE()	Returns current system date	SELECT CURRENT_DATE() AS current_date
CURRENT_TIME()	Returns current system time	SELECT CURRENT_TIME() AS current_time
EXTRACT()	Extracts date parts like WEEKDAY, MONTH, YEAR	EXTRACT(WEEKDAY FROM order_date)

Example: Count orders by weekday

```
SELECT COUNT(*) AS orders_per_day,
       EXTRACT(WEEKDAY FROM order_date) AS order_day
FROM orders
GROUP BY order_day
ORDER BY orders_per_day DESC;
```

5. Aggregations, Grouping, & Sorting 📊

- GROUP BY ALL automatically groups by all non-aggregated selected columns.
- Simplifies queries by avoiding manual listing of all grouping columns.

Example: Calculate revenue by pizza size per month

```
SELECT EXTRACT(MONTH FROM order_date) AS order_month,
       p.pizza_size,
       SUM(p.price * od.quantity) AS revenue
FROM orders o
INNER JOIN order_details od USING(order_id)
INNER JOIN pizzas p USING(pizza_id)
GROUP BY ALL
ORDER BY revenue DESC;
```

6. Joins & Relational Queries 🔗

NATURAL JOIN

- Joins tables on all columns with matching names.
- Avoids specifying join conditions explicitly.

Example: Find top revenue pizza category

```
SELECT pt.category,
       SUM(p.price * od.quantity) AS total_revenue
FROM order_details od
NATURAL JOIN pizzas p
NATURAL JOIN pizza_type pt
GROUP BY pt.category
ORDER BY total_revenue DESC
LIMIT 1;
```

Combination of LEFT JOIN, RIGHT JOIN, and NATURAL JOIN

Example: Analyze pizza popularity and revenue by name

```
SELECT COUNT(o.order_id) AS total_orders,
       AVG(p.price) AS average_price,
       SUM(p.price * od.quantity) AS total_revenue,
       pt.name AS pizza_name
FROM orders o
LEFT JOIN order_details od ON o.order_id = od.order_id
RIGHT JOIN pizzas p ON od.pizza_id = p.pizza_id
NATURAL JOIN pizza_type pt
```

```
GROUP BY pt.name, pt.category
ORDER BY total_revenue DESC, total_orders DESC;
```

7. Advanced CTEs & Optimization 🧩

Subqueries in HAVING Clause

- Filter grouped results by comparing to overall averages.
- Example: Select pizza types ordered less than average order quantity.

```
SELECT pt.name, pt.category, SUM(od.quantity) AS total_orders
FROM pizza_type pt
JOIN pizzas p ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od ON p.pizza_id = od.pizza_id
GROUP BY ALL
HAVING SUM(od.quantity) < (
  SELECT AVG(total_quantity)
  FROM (
    SELECT SUM(od.quantity) AS total_quantity
    FROM pizzas p
    JOIN order_details od ON p.pizza_id = od.pizza_id
    GROUP BY p.pizza_id
  ) AS subquery
);
```

Common Table Expressions (CTEs) & UNION ALL

- Use WITH to modularize queries.
- Combine results with UNION ALL.

Example: Identify the most ordered and cheapest pizza

```
WITH most_ordered AS (
  SELECT pizza_id, SUM(quantity) AS total_qty
  FROM order_details
  GROUP BY pizza_id
  ORDER BY total_qty DESC
  LIMIT 1
),
cheapest_pizza AS (
  SELECT pizza_id, price
  FROM pizzas
  WHERE price = (SELECT MIN(price) FROM pizzas)
  LIMIT 1
)

SELECT pizza_id, 'Most Ordered' AS Description, total_qty AS metric FROM
most_ordered
UNION ALL
SELECT pizza_id, 'Cheapest' AS Description, price AS metric FROM
cheapest_pizza;
```

Early Filtering for Performance

- Apply WHERE clauses inside CTEs before joins to reduce data size and improve query speed.

Example: Filter orders after a date for "Veggie" pizzas

```
WITH filtered_orders AS (
  SELECT order_id, order_date FROM orders WHERE order_date > '2015-11-01'
),
filtered_pizza_type AS (
  SELECT name, pizza_type_id FROM pizza_type WHERE category = 'Veggie'
)

SELECT fo.order_id, fo.order_date, fpt.name, od.quantity
FROM filtered_orders fo
JOIN order_details od ON fo.order_id = od.order_id
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN filtered_pizza_type fpt ON p.pizza_type_id = fpt.pizza_type_id;
```

---

8. Querying Semi-Structured JSON Data (VARIANT) 📦

Accessing Nested JSON Keys

- Use : operator to access keys inside VARIANT columns.

Example: Extract weekend hours from nested JSON

```
SELECT name, review_count, hours:Saturday, hours:Sunday
FROM yelp_business_data
WHERE categories LIKE '%Restaurant%'
  AND (hours:Saturday IS NOT NULL AND hours:Sunday IS NOT NULL)
  AND city = 'Philadelphia'
  AND stars = 5
ORDER BY review_count DESC;
```

Filtering on Nested JSON Attributes

- Use ILIKE for case-insensitive matching on JSON string values inside VARIANT.

Example: Find restaurants allowing dogs and accepting credit cards

```
SELECT business_id, name
FROM yelp_business_data
WHERE categories ILIKE '%Restaurant%'
  AND attributes:DogsAllowed ILIKE '%True%'
  AND attributes:BusinessAcceptsCreditCards ILIKE '%True%'
  AND city ILIKE '%Philadelphia%'
  AND stars = 5;
```